

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 2
по дисциплине: Исследование операций
по теме: Симплекс-метод в чистом виде

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Вирченко Ю.В.

Белгород 2024

Цель работы: изучение симплекс-метода для решения задачи линейного программирования с использованием симплекс-таблиц, получение навыков кодирования изученного алгоритма, отладки и тестирования соответствующих программ.

Задания для подготовки к работе

1. Выяснить: какой вид должна иметь задача ЛП, чтобы можно было применять симплекс-метод в чистом виде, а также как составляется первая симплекс-таблица?
2. Изучить алгоритм перехода от одной симплекс-таблицы к другой при решении задачи симплекс-методом.
3. Запрограммировать и отладить изученный алгоритм. В рамках подготовки тестовых данных решить вручную одну из следующих ниже задач.

Вариант 7

7.

$$\begin{aligned} z &= 2x_1 + 3x_2 - x_4 \rightarrow \max; \\ \begin{cases} 2x_1 - x_2 - 2x_4 + x_5 = 16, \\ 3x_1 + 2x_2 + x_3 - 3x_4 = 18, \\ -x_1 + 3x_2 + 4x_4 + x_6 = 24, \end{cases} \\ x_i &\geq 0 \quad (i = \overline{1,6}). \end{aligned}$$

Ручное решение:

Лабораторная работа №2

$$Z = 2x_1 + 3x_2 - x_4 \rightarrow \max$$

$$\begin{cases} 2x_1 - x_2 - 2x_4 + x_5 = 16 \\ 3x_1 + 2x_2 + x_3 - 3x_4 = 18 \\ -x_1 + 3x_2 + 4x_4 + x_6 = 24 \end{cases}$$

$$x_i \geq 0 \quad (i = \overline{1,6})$$

$$Z - 2x_1 - 3x_2 + x_4 = 0$$

Базис	x_1	x_2	x_3	x_4	x_5	x_6	b_i	$b_i / \text{разреш. элемент}$
x_5	2	-1	0	-2	1	0	16	$16 : -1 < 0$
x_3	3	2	1	-3	0	0	18	$18 : 2 = 9$
x_6	-1	(3)	0	4	0	1	24	$24 : 3 = 8 \leftarrow \min$
	-2	-3	0	1	0	0	0	
		$\uparrow$						$\min$

Это отрицательное значение - мин

и оптимальный

Находим наименьший элемент в строке

или столбце и оставляем эту строку без изменений

На пересечение минимальных значений получено значение в столбце

Базис	x_1	x_2	x_3	x_4	x_5	x_6	b_i	$b_i / \text{разреш.}$
x_5	$\frac{5}{3}$	0	0	$-\frac{2}{3}$	1	$\frac{1}{3}$	24	$\frac{72}{5}$
x_3	$\frac{11}{3}$	0	1	$-\frac{17}{3}$	0	$-\frac{2}{3}$	2	$\frac{6}{11} \leftarrow \min$
x_2	$-\frac{1}{3}$	1	0	$\frac{4}{3}$	0	$\frac{1}{3}$	8	< 0
	-3	0	0	5	0	1	24	
	$\uparrow \min$							

Действия: 1) Делю 3 строку на 3 для удобства, 2) Складываю 3 строку и первую чтобы получить 0 в столбце

Для получения 0 во второй строке

1) умножаю 3 строку на 2, 2) складываю от второй строки

Получаем нули в столбце

Если отрицательный элемент $\rightarrow$ минимальный

Размер	x_1	x_2	x_3	x_4	x_5	x_6	b_i	$b_i / \text{погреш. элемент}$
x_5	0	0	$-\frac{5}{11}$	$\frac{21}{11}$	1	$\frac{7}{11}$	$\frac{254}{11}$	
x_1	1	0	$\frac{3}{11}$	$-\frac{17}{11}$	0	$-\frac{2}{11}$	$\frac{6}{11}$	
x_2	0	1	$\frac{1}{11}$	$\frac{9}{11}$	0	$\frac{3}{11}$	$\frac{6}{11}$	0
	0	0	$\frac{9}{11}$	$\frac{4}{11}$	0	$\frac{5}{11}$	$\frac{282}{11}$	

Вторую строку умножаем на ~~$\frac{11}{3}$~~ $\frac{3}{11}$ для упрощения

Действия: 1) умножить II строку на $\frac{5}{3}$ и 2) отнять от I строки, чтобы получить нуль

1) умножить II строку на $\frac{1}{3}$ и

2) сложившая с III строкой, чтобы получить нуль в столбце

В последней строке нет опор. элемент $\Rightarrow$ нет оптимальной.

$$x_1 = \frac{6}{11}$$

$$x_2 = \frac{90}{11}$$

$$x_3 = 0$$

$$x_4 = 0$$

$$x_5 = \frac{254}{11}$$

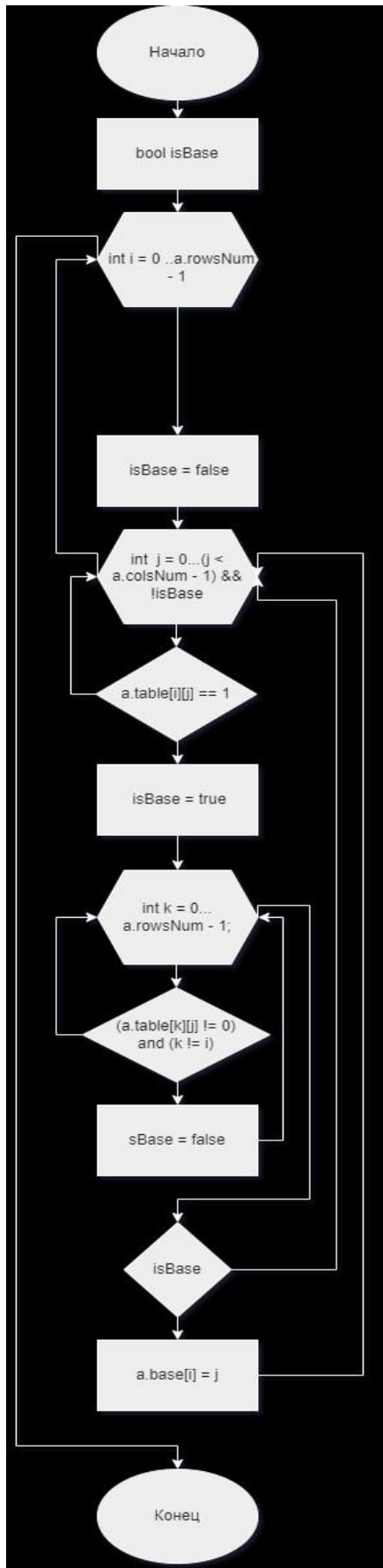
$$x_6 = 0$$

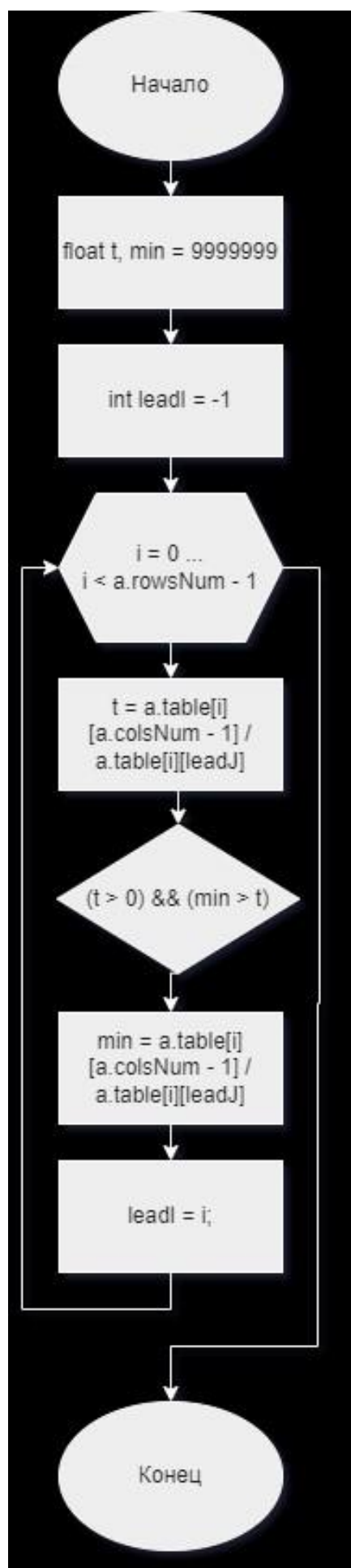
$$\Rightarrow z = \frac{282}{11} = 26$$

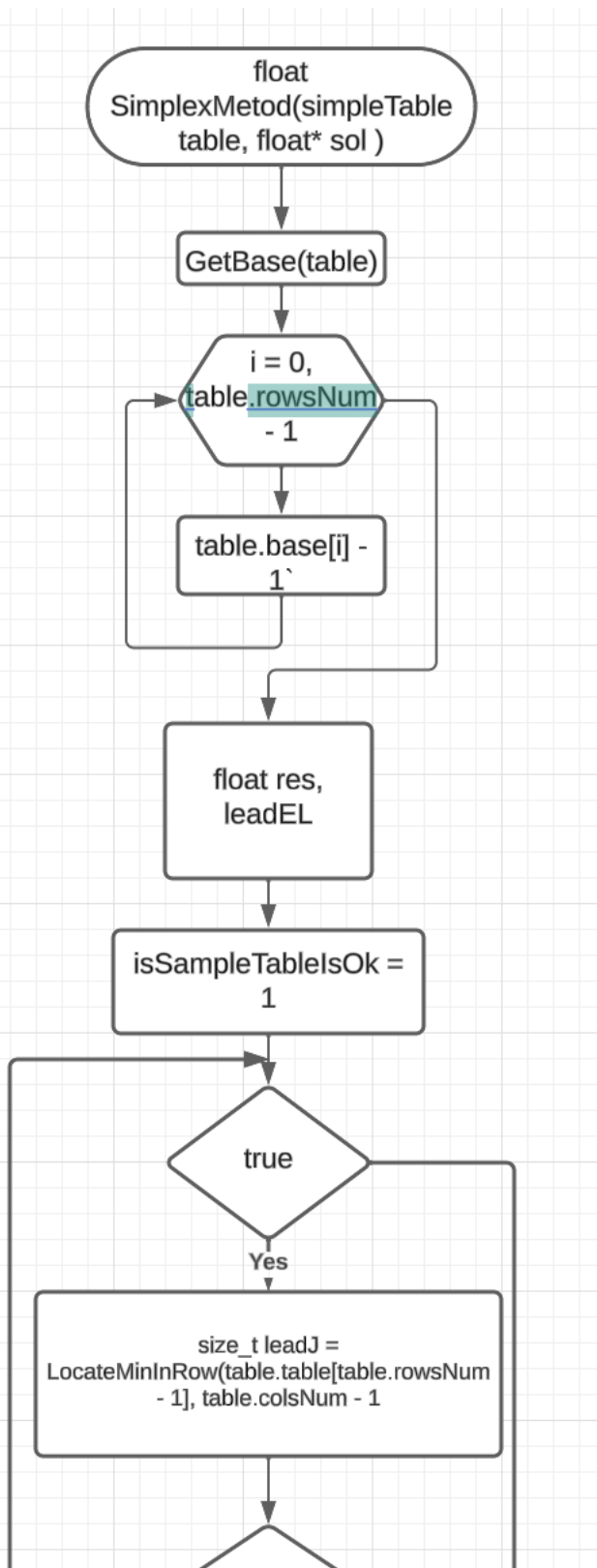
Answer: $z = 26$

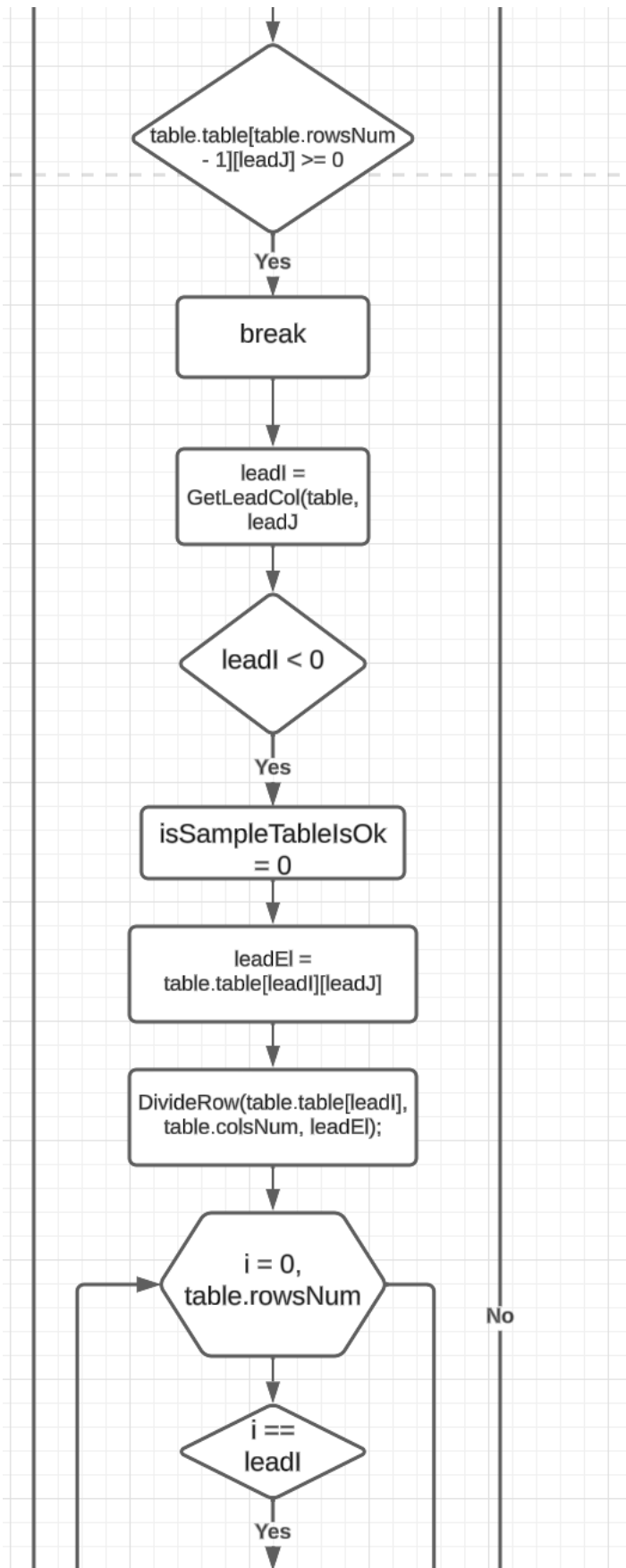
Блок-схемы:

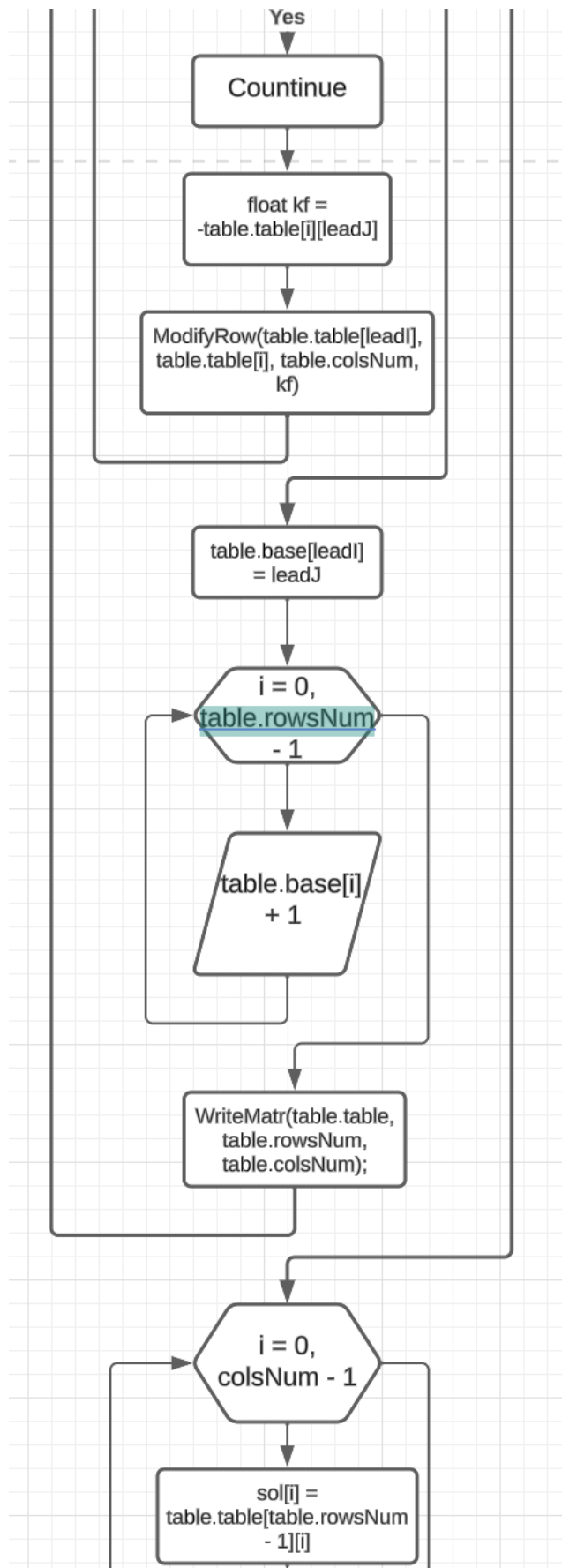
GetBase

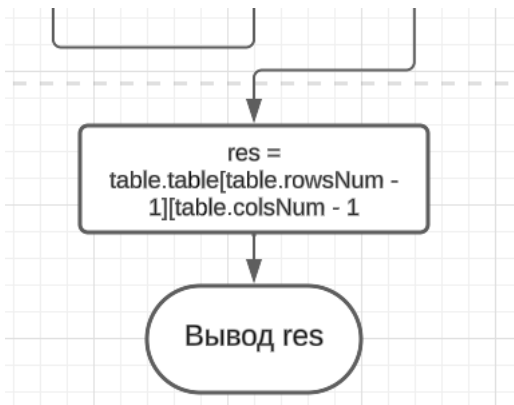












Программная реализация:

```

#include <iostream>
#include <locale>
#include <cmath>

using namespace std;

typedef float** matr;

typedef struct goalFunc {
    float* func;
    bool toMax;
} goalFunc;

typedef struct simpTable {
    matr table;
    unsigned* base;
    size_t rowsNum;
    size_t colsNum;
} simpTable;

void CreateMatr(matr* a, size_t n, size_t m) {
    *a = new float* [n];
    for (size_t i = 0; i < n; i++)
        (*a)[i] = new float[m];
}

void CreateTable(simpTable* a, size_t n, size_t m) {
    (*a).base = new unsigned[n];
    CreateMatr(&((*a).table), n + 1, m + 1);
    (*a).rowsNum = n + 1;
    (*a).colsNum = m;
}

//ввод матрицы a размера nxm
void ReadMatr(matr a, size_t n, size_t m) {
    for (size_t i = 0; i < n; i++)
        for (size_t j = 0; j < m; j++)
            cin >> a[i][j];
}

void GetBase(simpTable a) {
    bool isBase;
    for (size_t i = 0; i < a.rowsNum - 1; i++) {
        isBase = false;
    }
  
```

```

        for (size_t j = 0; (j < a.colsNum - 1) && !isBase; j++) {
            if (a.table[i][j] == 1) {
                isBase = true;
                for (size_t k = 0; k < a.rowsNum - 1; k++) {
                    if ((a.table[k][j] != 0) and (k != i))
                        isBase = false;
                }
                if (isBase)
                    a.base[i] = j;
            }
        }
    }
}

```

```

//вывод матрицы a размера nxm
void WriteMatr(matr a, size_t n, size_t m) {
    for (size_t j = 0; j < m - 1; j++)
        cout << "x" << j + 1 << ' ';
    cout << " | " << "z\n";
    for (size_t i = 0; i < n; i++) {
        for (size_t j = 0; j < m - 1; j++)
            cout << a[i][j] << ' ';
        cout << " | " << a[i][m - 1] << '\n';
    }
    cout << '\n';
}

```

```

void WriteSolution(float* sol, size_t n) {
    for (size_t j = 0; j < n; j++)
        cout << sol[j] << "x" << j + 1 << " + ";
    cout << "\b";
}

```

```

//обмен чисел
void Swap(float* a, float* b) {
    float t = *a;
    *a = *b;
    *b = t;
}

```

```

//обмен массивов
void SwapArr(float* a, float* b, size_t n) {
    for (size_t i = 0; i < n; i++)
        Swap(&a[i], &b[i]);
}

```

```

//прибавляет к строке b строку a, умноженную на коэффициент k
void ModifyRow(float* a, float* b, size_t n, float k) {
    for (size_t i = 0; i < n; i++) {
        b[i] += a[i] * k;
        if (abs(b[i]) <= 0.0001)
            b[i] = 0;
    }
}

```

```

void CopyMatr(matr a, size_t n, size_t m, matr res) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            res[i][j] = a[i][j];
}

```

```

void WriteArray(int C[100], int k) {

```

```

    for (size_t i = 0; i < k; i++)
        cout << C[i] + 1 << ' ';
    cout << endl;
}

//деление строки на число
void DivideRow(float* a, size_t n, float k) {
    for (size_t i = 0; i < n; i++) {
        a[i] /= k;
        if (abs(a[i]) <= 0.0001)
            a[i] = 0;
    }
}

size_t LocateMinInRow(float* a, size_t n) {
    float min = a[0];
    size_t minI = 0;
    for (size_t i = 1; i < n; i++)
        if (min > a[i]) {
            min = a[i];
            minI = i;
        }
    return minI;
}

int GetLeadCol(simpTable a, size_t leadJ) {
    float t, min = 9999999;
    int leadI = -1;
    for (size_t i = 0; i < a.rowsNum - 1; i++) {
        t = a.table[i][a.colsNum - 1] / a.table[i][leadJ];
        if ((t > 0) && (min > t)) {
            min = a.table[i][a.colsNum - 1] / a.table[i][leadJ];
            leadI = i;
        }
    }
    return leadI;
}

void DeleteMatr(matr a, size_t n) {
    for (size_t i = 0; i < n; i++)
        delete a[i];
    delete a;
}

int isSimpTableOk;

float SimplexMethod(simpTable table, float* sol) {
    GetBase(table);
    cout << "Базисы: ";
    for (size_t i = 0; i < table.rowsNum - 1; i++)
        cout << 'x' << table.base[i] + 1 << ' ';
    cout << '\n';
    float res, leadEl;
    isSimpTableOk = 1;
    while (true) {
        size_t leadJ = LocateMinInRow(table.table[table.rowsNum - 1], table.colsNum
- 1);
        if (table.table[table.rowsNum - 1][leadJ] >= 0)
            break;
        int leadI = GetLeadCol(table, leadJ);
        if (leadI < 0) {
            cout << "Функция не ограничена на множестве допустимых значений.\n";
            isSimpTableOk = 0;
            break;
        }
    }
}

```

```

    }
    leadEl = table.table[leadI][leadJ];
    DivideRow(table.table[leadI], table.colsNum, leadEl);
    for (size_t i = 0; i < table.rowsNum; i++) {
        if (i == leadI)
            continue;
        float kf = -table.table[i][leadJ];
        ModifyRow(table.table[leadI], table.table[i], table.colsNum, kf);
    }
    table.base[leadI] = leadJ;
    cout << "Базисы: ";
    for (size_t i = 0; i < table.rowsNum - 1; i++)
        cout << 'x' << table.base[i] + 1 << ' ';
    cout << '\n';
    WriteMatr(table.table, table.rowsNum, table.colsNum);
}
for (size_t i = 0; i < table.colsNum - 1; i++)
    sol[i] = table.table[table.rowsNum - 1][i];
res = table.table[table.rowsNum - 1][table.colsNum - 1];
return res;
}

int main() {
    cout.precision(2);
    size_t n, m;
    setlocale(LC_ALL, "RUS");
    cout << "Введите два числа n и m - размеры матрицы системы ограничений: ";
    cin >> n >> m;

    simpTable table;
    CreateTable(&table, n, m);
    float* sol = new float[m - 1];
    n++;

    cout << "Введите матрицу системы ограничений и строку целевой функции:\n";
    ReadMatr(table.table, n, m);

    cout << "Введённая матрица:\n";
    WriteMatr(table.table, n, m);

    float res = SimplexMethod(table, sol);
    if (isSimpTableOk) {
        cout << res;
    }
}

```

```
Консоль отладки Microsoft Visual Studio

x1 x2 x3 x4 x5 x6 | z
2 -1 0 -2 1 0 | 16
3 2 1 -3 0 0 | 18
-1 3 0 4 0 1 | 24
-2 -3 0 1 0 0 | 0

Базисы: x5 x3 x6
Базисы: x5 x3 x2
x1 x2 x3 x4 x5 x6 | z
1.7 0 0 -0.67 1 0.33 | 24
3.7 0 1 -5.7 0 -0.67 | 2
-0.33 1 0 1.3 0 0.33 | 8
-3 0 0 5 0 1 | 24

Базисы: x5 x1 x2
x1 x2 x3 x4 x5 x6 | z
0 0 -0.45 1.9 1 0.64 | 23
1 0 0.27 -1.5 0 -0.18 | 0.55
0 1 0.091 0.82 0 0.27 | 8.2
0 0 0.82 0.36 0 0.45 | 26

26
C:\Users\vovad\source\repos\IO2\x64\Debug\IO2.exe
Нажмите любую клавишу, чтобы закрыть это окно: _
```

Вывод: в ходе выполнения лабораторной работы был изучен симплекс-метод для решения задачи линейного программирования с использованием симплекс-таблиц, так же были получены навыки кодирования изученного алгоритма, отладки и тестирования соответствующих программ. Что ручной метод, что программный выдаёт одинаковые ответы, из чего можно сделать вывод, что алгоритм решения правильный.